

The Codex of C

Written for MNGL — forged in logic and memory.

1 What is C?

C is the root language. Every other modern language — from Python to Rust — carries its DNA. It is not forgiving. It assumes you know what you are doing. But in exchange, it gives you control over the machine itself.

2 Program Structure

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

Notes:

- `#include` brings in header files.
- `argc`, `argv[]` handle command line arguments.

3 Data Types and Operators

3.1 Primitives

<code>int</code>	4 bytes (whole numbers)
<code>float</code>	4 bytes (decimal)
<code>double</code>	8 bytes (high precision)
<code>char</code>	1 byte (single character)
<code>void</code>	no type, used in functions/pointers

3.2 Operators

- Arithmetic: `+` `-` `*` `/` `%`
- Comparison: `==` `!=` `>` `<` `>=` `<=`
- Logical: `&&` `||` `!`
- Bitwise: `&` `|` `^` `<<` `>>` `~`
- Increment/Decrement: `++` `--`

4 Casting and Type Conversion

```
float f = 10.5;
int i = (int)f; // explicit cast
```

5 Constants, Macros, and Enums

```
#define PI 3.14159
#define SQUARE(x) ((x)*(x))

enum Day { MON, TUE, WED, THU, FRI, SAT, SUN };

int main() {
    printf("%f\n", PI);
    printf("%d\n", SQUARE(5));
    printf("Day: %d\n", FRI);
}
```

6 Arrays and Pointers

```
int arr[5] = {1,2,3,4,5};
int *ptr = arr; // arr = &arr[0]
printf("%d\n", *(ptr + 2)); // prints 3
```

Arrays decay into pointers, and memory is contiguous.

7 Pointers to Pointers

```
int x = 42;
int *p = &x;
int **pp = &p;

printf("%d\n", **pp);
```

8 Dynamic Memory

```
#include <stdlib.h>

int *arr = malloc(5 * sizeof(int));
if (arr == NULL) return 1;

for (int i = 0; i < 5; i++) arr[i] = i*i;
free(arr);
```

Always free what you allocate. If you forget — you leak memory.

9 Structures and Typedefs

```
typedef struct {
    char name[50];
    int level;
    float power;
} Player;

int main() {
    Player p = {"Guts", 99, 9999.9};
    printf("%s is level %d\n", p.name, p.level);
}
```

10 Linked Lists (Intro)

```
typedef struct Node {
    int value;
    struct Node *next;
} Node;

int main() {
    Node a = {1, NULL};
    Node b = {2, NULL};
    a.next = &b;
    printf("%d -> %d\n", a.value, a.next->value);
}
```

11 File I/O — Text and Binary

11.1 Text Files

```
FILE *f = fopen("data.txt", "w");
fprintf(f, "MNGL %d\n", 184);
fclose(f);
```

11.2 Reading Binary Data

```
int nums[3] = {1,2,3};
FILE *f = fopen("nums.bin", "wb");
fwrite(nums, sizeof(int), 3, f);
fclose(f);
```

12 Header Files and Modularity

Split code into separate files:

```
// math_utils.h
int add(int a, int b);

// math_utils.c
#include "math_utils.h"
int add(int a, int b) { return a + b; }

// main.c
#include <stdio.h>
#include "math_utils.h"
int main() { printf("%d", add(2,3)); }
```

13 Command Line Arguments

```
int main(int argc, char *argv[]) {
    for (int i = 0; i < argc; i++)
        printf("arg[%d] = %s\n", i, argv[i]);
}
```

14 Error Handling

```
FILE *f = fopen("data.txt", "r");
if (!f) {
    perror("File error");
    return 1;
}
```

15 Debugging and Segfaults

- Use printf() tracing.
- Run with gdb ./program.
- Segfault = you touched memory that isn't yours.

16 Memory Model

Stack = local vars, automatic. Heap = malloc/free, manual. Globals = static, fixed.
Code = text segment (read-only).

17 Preprocessor Magic

#define, #ifdef, #ifndef control conditional compilation.

```
#ifdef DEBUG
printf("Debug mode on\n");
#endif
```

18 Mini Project Example: Simple Database

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char name[20];
    int score;
} Student;

int main() {
    FILE *f = fopen("students.txt", "w");
    Student s[2] = {{ "Ilyas", 95 }, { "Rime", 99 }};
    for (int i = 0; i < 2; i++)
        fprintf(f, "%s %d\n", s[i].name, s[i].score);
    fclose(f);
}
```

19 Final Philosophy

C is not a language; it's a negotiation with the machine. It rewards precision and punishes laziness. Every pointer is a blade — sharp, elegant, and deadly if mishandled. But when mastered, it turns you into an architect of logic, a sculptor of electrons.

"In memory we trust. In logic we forge."